

## GoITech Solutions S.A.

### Plan de Mantenimiento

Documento de cierre del proyecto | Ingeniería de Software

|                                  |                                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------|
| <b>Responsable del documento</b> | Jenny Sofía Morales López                                                              |
| <b>Sistema</b>                   | Sistema web de apuestas - Plataforma de predicciones deportivas<br>FIFA World Cup 2026 |
| <b>Versión</b>                   | 1.0                                                                                    |
| <b>Fecha</b>                     | Mayo de 2026                                                                           |
| <b>Estado</b>                    | Aprobado                                                                               |

## Contenido

|                                                             |    |
|-------------------------------------------------------------|----|
| 1. Introducción.....                                        | 3  |
| 2. Objetivo del plan de mantenimiento .....                 | 4  |
| 2.1 Objetivos específicos.....                              | 4  |
| 3. Alcance del mantenimiento.....                           | 4  |
| 4. Contexto técnico del sistema.....                        | 4  |
| 5. Roles y responsables .....                               | 5  |
| 6. Clasificación de prioridades.....                        | 5  |
| 7. Mantenimiento correctivo .....                           | 6  |
| 7.1 Procedimiento correctivo.....                           | 6  |
| 8. Mantenimiento adaptativo.....                            | 7  |
| 8.1 Procedimiento adaptativo.....                           | 7  |
| 9. Mantenimiento perfectivo .....                           | 7  |
| 9.1 Procedimiento perfectivo.....                           | 7  |
| 10. Mantenimiento preventivo.....                           | 8  |
| 10.1 Actividades preventivas obligatorias.....              | 8  |
| 10.2 Consultas preventivas sugeridas .....                  | 9  |
| 11. Procedimiento de control de cambios.....                | 9  |
| 12. Plan de respaldo, restauración y continuidad .....      | 9  |
| 12.1 Procedimiento de restauración .....                    | 10 |
| 13. Seguridad, auditoría y trazabilidad .....               | 10 |
| 13.1 Manejo de credenciales .....                           | 10 |
| 14. Indicadores de seguimiento.....                         | 11 |
| 15. Matriz de periodicidad .....                            | 11 |
| 16. Registro de actividades de mantenimiento .....          | 12 |
| 17. Riesgos operativos y acciones de mitigación.....        | 14 |
| 18. Criterios de aceptación posterior al mantenimiento..... | 14 |
| 19. Recomendaciones.....                                    | 14 |
| 20. Referencias .....                                       | 15 |

## 1. Introducción

Sistema web de apuestas es una plataforma web de predicciones deportivas para la Copa Mundial FIFA 2026. El sistema integra autenticación, gestión de usuarios, ligas, invitaciones, vaticinios, clasificación, administración del torneo y distribución de premios. Por esa naturaleza, el mantenimiento no debe limitarse a “corregir errores”; debe asegurar continuidad operativa, integridad de datos, seguridad de credenciales, trazabilidad, actualización tecnológica y mejora continua.

Este documento define el plan de mantenimiento del sistema desde una perspectiva técnica, operativa y de seguridad. Se contemplan actividades correctivas, adaptativas, perfectivas y preventivas, indicando responsables, periodicidad, procedimientos, evidencias y criterios de aceptación. El enfoque está orientado a un sistema real desplegado en la nube, con frontend Angular, backend FastAPI y base de datos PostgreSQL.

## 2. Objetivo del plan de mantenimiento

Establecer un marco de trabajo claro para mantener el Sistema web de apuestas después de su implementación y despliegue, asegurando que las fallas se atiendan oportunamente, los cambios se gestionen de forma controlada, las mejoras se prioricen con base en valor de negocio y las actividades preventivas reduzcan riesgos de seguridad, indisponibilidad o pérdida de información.

### 2.1 Objetivos específicos

- Definir responsabilidades para mantenimiento de frontend, backend, base de datos, seguridad y pruebas.
- Establecer procedimientos para mantenimiento correctivo, adaptativo, perfectivo y preventivo.
- Proteger la continuidad del servicio mediante respaldos, revisión de logs y monitoreo de sesiones.
- Asegurar que los cambios se prueben antes de pasar a producción y queden documentados.
- Mantener evidencia verificable de actividades de mantenimiento, incidentes, cambios, despliegues y validaciones.
- Reducir riesgos asociados a credenciales, tokens, roles, auditoría, integridad de datos y disponibilidad del sistema.

## 3. Alcance del mantenimiento

El plan aplica a los componentes técnicos y funcionales que conforman Sistema web de apuestas:

| Componente               | Alcance de mantenimiento                                                                                                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Frontend Angular         | Pantallas, rutas, formularios, validaciones, estilos responsivos, mensajes de retroalimentación y comunicación con API.                |
| Backend FastAPI          | Endpoints REST, servicios de negocio, autenticación, autorización, JWT, recuperación de contraseña, invitaciones y reglas funcionales. |
| Base de datos PostgreSQL | Tablas, llaves foráneas, índices, constraints, soft delete, audit log, user sessions, respaldos y restauración.                        |
| Correo SMTP              | Envío de invitaciones, recuperación de contraseña, configuración SMTP y validación de entrega.                                         |
| Seguridad aplicativa     | Contraseñas cifradas con bcrypt, roles, protección de endpoints, tokens, permisos de BD y control de sesiones.                         |
| Despliegue               | Variables de entorno, URL pública, configuración de frontend/backend, conectividad con BD y operación en nube gratuita.                |
| Documentación            | Manual técnico, manual de usuario, bitácora de cambios, evidencia de pruebas y registros de mantenimiento.                             |

## 4. Contexto técnico del sistema

Para que el mantenimiento sea aplicable al sistema real, se toma como base la arquitectura implementada durante el proyecto. La plataforma se compone de una aplicación web, una API backend, una base de datos relacional y servicios externos de correo.

| Elemento | Tecnología / componente | Consideración de mantenimiento                                                                         |
|----------|-------------------------|--------------------------------------------------------------------------------------------------------|
| Frontend | Angular                 | Revisar compatibilidad de dependencias, rutas protegidas, responsividad, formularios y consumo de API. |
| Backend  | FastAPI con Python      | Revisar endpoints, servicios, validaciones, manejo de errores, documentación Swagger y seguridad.      |

| Elemento             | Tecnología / componente            | Consideración de mantenimiento                                                                |
|----------------------|------------------------------------|-----------------------------------------------------------------------------------------------|
| Base de datos        | PostgreSQL                         | Mantener integridad, respaldos, índices, constraints, soft delete, audit log y user_sessions. |
| Autenticación        | JWT + refresh token                | Verificar expiración, revocación de sesiones y protección de endpoints.                       |
| Contraseñas          | bcrypt                             | Asegurar que nunca se almacenen contraseñas en texto plano.                                   |
| Correo               | SMTP Gmail o proveedor equivalente | Validar credenciales de aplicación, variables de entorno y plantillas HTML.                   |
| Control de versiones | GitHub / GitLab                    | Usar ramas, commits descriptivos, pull requests y etiquetas de versión.                       |

## 5. Roles y responsables

El mantenimiento debe asignarse por área para evitar que los incidentes queden sin dueño. Aunque el equipo pueda ser pequeño, cada actividad necesita un responsable primario y un apoyo técnico.

| Rol                       | Responsabilidad principal                     | Actividades específicas                                                                 |
|---------------------------|-----------------------------------------------|-----------------------------------------------------------------------------------------|
| Líder de mantenimiento    | Coordinar, priorizar y aprobar cambios.       | Clasificar tickets, revisar impacto, aprobar despliegues y mantener la bitácora.        |
| Responsable backend       | Mantener API y lógica de negocio.             | Corregir endpoints, revisar logs, proteger rutas, validar servicios y manejar errores.  |
| Responsable frontend      | Mantener interfaz y experiencia de usuario.   | Ajustar vistas, validar formularios, responsividad, accesibilidad y mensajes visibles.  |
| Responsable base de datos | Mantener integridad, respaldos y rendimiento. | Revisar índices, constraints, audit log, user_sessions, soft delete y restauración.     |
| Responsable de seguridad  | Revisar controles de acceso y credenciales.   | Validar bcrypt, roles, variables de entorno, permisos mínimos y exposición de secretos. |
| Responsable QA            | Ejecutar pruebas antes de cada liberación.    | Aplicar pruebas de regresión, aceptación, casos críticos y evidencia de resultados.     |

| Actividad                     | Backend | Frontend | BD | Seguridad | QA | Líder |
|-------------------------------|---------|----------|----|-----------|----|-------|
| Corrección de bug de login    | R       | C        | C  | C         | A  | A     |
| Cambio visual en dashboard    | C       | R        | I  | I         | A  | A     |
| Backup y restauración de BD   | C       | I        | R  | A         | C  | A     |
| Actualización de dependencias | R       | R        | I  | A         | A  | A     |
| Revisión de audit_log         | C       | I        | R  | A         | C  | A     |
| Despliegue a producción       | R       | R        | C  | A         | A  | A     |

Leyenda: R = responsable de ejecutar, A = Aprueba, C = Consultado, I = Informado.

## 6. Clasificación de prioridades

Todo mantenimiento debe clasificarse según impacto y urgencia. Esta clasificación ayuda a decidir qué se atiende primero y qué puede planificarse para un sprint posterior.

| Prioridad | Descripción                                                         | Ejemplos en Sistema web de apuestas                                                                                            | Tiempo objetivo                                                  |
|-----------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Crítica   | Afecta seguridad, disponibilidad total o pérdida de datos.          | No se puede iniciar sesión; tokens expuestos; base de datos inaccesible; cálculo de premios incorrecto al cierre.              | Atención inmediata; corrección o mitigación en menos de 8 horas. |
| Alta      | Afecta funciones centrales, pero el sistema aún opera parcialmente. | Falla recuperación de contraseña; no se registran sesiones; errores en invitaciones; jugadores no pueden registrar vaticinios. | 24 horas.                                                        |
| Media     | Afecta usabilidad o una función no crítica.                         | Mensaje visual no aparece; filtro de usuarios falla; vista móvil con detalles menores.                                         | 48 a 72 horas.                                                   |
| Baja      | Mejora estética, ajuste menor o solicitud futura.                   | Cambio de color, texto de ayuda, mejora de iconografía, orden de columnas.                                                     | Próximo sprint o release planificado.                            |

## 7. Mantenimiento correctivo

El mantenimiento correctivo se aplica cuando una funcionalidad existente falla o se comporta distinto a lo esperado. Su objetivo es restaurar el funcionamiento normal del sistema con el menor impacto posible para los usuarios.

### 7.1 Procedimiento correctivo

1. Registrar el incidente en el tablero del proyecto con fecha, responsable, módulo afectado y evidencia.
2. Clasificar prioridad según impacto: crítica, alta, media o baja.
3. Reproducir el error en ambiente local o de pruebas.
4. Identificar si la causa está en *frontend*, *backend*, base de datos, integración SMTP o despliegue.
5. Crear una rama de corrección en Git con nombre descriptivo, por ejemplo `fix/logout-session-state`.
6. Aplicar la corrección, ejecutar pruebas unitarias o funcionales y validar en Swagger cuando aplique.
7. Registrar evidencia de antes y después: captura, log, consulta SQL o caso de prueba.
8. Hacer merge controlado, desplegar y verificar en producción.
9. Cerrar el incidente documentando causa raíz y solución aplicada.

| Incidente correctivo                                     | Diagnóstico mínimo                                                                                                                                   | Evidencia requerida                                                                |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Login devuelve 401 aunque las credenciales son correctas | Revisar <code>usuario.estado</code> , <code>password_hash</code> , endpoint <code>/api/auth/login</code> y función <code>verificar_password</code> . | Log backend, captura de login, consulta a tabla <code>usuario</code> .             |
| Logout no cambia sesión a cerrada                        | Verificar endpoint <code>/api/auth/logout</code> , <code>refresh_token</code> enviado y actualización de <code>user_sessions</code> .                | Log POST logout, consulta <code>user_sessions</code> antes/después.                |
| Recuperación de contraseña no abre pantalla              | Validar <code>FRONTEND_URL</code> , ruta <code>/reset-password</code> y token generado.                                                              | Correo recibido, URL, pantalla Angular y tabla <code>password_reset_token</code> . |

| Incidente correctivo                    | Diagnóstico mínimo                                                    | Evidencia requerida                                                          |
|-----------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------|
| Usuario jugador accede a administración | Revisar validación de rol en frontend y backend.                      | Perfil del usuario, intento a /admin/usuarios y respuesta 403 o redirección. |
| Baja lógica no se refleja               | Validar endpoint PATCH usuarios, estado, deleted_at y permisos admin. | Consulta SQL mostrando estado inactivo y deleted_at.                         |

## 8. Mantenimiento adaptativo

El mantenimiento adaptativo permite que Sistema web de apuestas siga funcionando cuando cambian condiciones externas: proveedores, reglas de negocio, infraestructura, versiones de navegador, servicios de correo o plataforma de despliegue.

### 8.1 Procedimiento adaptativo

10. Identificar el cambio externo y documentar la fecha en que será necesario aplicarlo.
11. Analizar impacto sobre módulos, base de datos, variables de entorno y documentación.
12. Definir alternativa técnica y estimar esfuerzo.
13. Implementar en ambiente de pruebas.
14. Ejecutar pruebas de regresión sobre los flujos afectados.
15. Actualizar Manual Técnico, .env.example y bitácora de cambios.
16. Desplegar y monitorear durante las primeras 24 horas.

| Cambio externo                        | Impacto esperado                                         | Acción adaptativa                                                            |
|---------------------------------------|----------------------------------------------------------|------------------------------------------------------------------------------|
| Cambio de proveedor SMTP              | Puede afectar invitaciones y recuperación de contraseña. | Actualizar variables SMTP, probar envío real y validar plantillas HTML.      |
| Cambio de URL pública del frontend    | Links de correos podrían apuntar a una URL incorrecta.   | Actualizar FRONTEND_URL y reenviar pruebas de recuperación e invitación.     |
| Migración de BD local a nube          | Riesgo de permisos, latencia y conexión.                 | Actualizar DB_HOST, roles, backups, SSL y pruebas de conexión.               |
| Cambio en calendario del Mundial 2026 | Afecta partidos, horarios y cierre de vaticinios.        | Actualizar datos de partidos y validar reglas de cierre 15 minutos antes.    |
| Nueva versión de Angular o FastAPI    | Puede romper dependencias o sintaxis.                    | Actualizar dependencias de forma controlada y ejecutar pruebas de regresión. |

## 9. Mantenimiento perfectivo

El mantenimiento perfectivo busca mejorar el sistema sin que exista una falla directa. Estas mejoras deben responder a valor para usuarios, administradores o equipo técnico, evitando cambios improvisados que compliquen la estabilidad del sistema.

### 9.1 Procedimiento perfectivo

17. Registrar la mejora como historia de usuario o solicitud técnica.
18. Definir objetivo, usuario beneficiado y criterio de aceptación.
19. Priorizar en el backlog según valor, esfuerzo y riesgo.

20. Diseñar la mejora respetando la paleta, componentes y flujo UX existente.
21. Implementar en rama independiente.
22. Ejecutar pruebas funcionales y revisión visual en resoluciones 360px, 768px y 1280px.
23. Publicar la mejora en un release menor o planificado.

| Mejora perfecta                           | Beneficio                                                               | Criterio de aceptación                                                  |
|-------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Agregar filtros en gestión de usuarios    | Facilita encontrar usuarios activos, inactivos o por rol.               | El administrador puede filtrar por rol y estado sin recargar la página. |
| Mejorar dashboard del jugador             | Permite ver información relevante de ligas, puntos y próximos partidos. | El dashboard muestra datos reales y mantiene tiempo de carga aceptable. |
| Agregar indicador de carga en formularios | Evita que el usuario repita acciones por incertidumbre.                 | Cada acción asíncrona muestra estado procesando y mensaje final.        |
| Optimizar vista móvil de perfil           | Mejora uso desde teléfonos.                                             | La pantalla es usable en 360px sin traslapes ni scroll horizontal.      |
| Agregar búsqueda pública de ligas         | Mejora descubrimiento de ligas.                                         | El usuario puede buscar por nombre de liga y ver resultados relevantes. |

## 10. Mantenimiento preventivo

El mantenimiento preventivo es el más importante para evitar fallas futuras. En Sistema web de apuestas se enfoca en seguridad, disponibilidad, datos, dependencias, sesiones y trazabilidad.

### 10.1 Actividades preventivas obligatorias

| Actividad                                                | Periodicidad                             | Responsable                     | Evidencia                                                |
|----------------------------------------------------------|------------------------------------------|---------------------------------|----------------------------------------------------------|
| Revisión de sesiones activas y cerradas en user_sessions | Semanal                                  | Responsable backend / seguridad | Consulta SQL con sesiones activas, cerradas y expiradas. |
| Verificación de contraseñas cifradas                     | Mensual y antes de demo                  | Responsable seguridad           | Consulta mostrando password_hash con bcrypt.             |
| Backup de PostgreSQL                                     | Diario en producción; semanal en pruebas | Responsable BD                  | Archivo .sql o dump con fecha y responsable.             |
| Prueba de restauración de backup                         | Mensual                                  | Responsable BD + QA             | Evidencia de restauración en ambiente de prueba.         |
| Revisión de audit_log                                    | Semanal                                  | Responsable BD / seguridad      | Muestra de registros INSERT, UPDATE, DELETE.             |
| Revisión de dependencias npm/pip                         | Mensual                                  | Frontend / backend              | Reporte npm audit o pip list con acciones tomadas.       |
| Revisión de variables de entorno                         | Antes de cada release                    | Seguridad                       | .env.example actualizado y sin secretos en repositorio.  |
| Pruebas de regresión M1-M7                               | Antes de cada despliegue                 | QA                              | Matriz de pruebas ejecutada con resultado.               |



## 10.2 Consultas preventivas sugeridas

Las siguientes consultas sirven como guía para revisar la salud básica del sistema durante mantenimiento:

**Sesiones abiertas:** `SELECT id_session, id_usuario, fecha_creacion, fecha_expiracion, estado, revocada FROM user_sessions WHERE estado = 'activa' ORDER BY fecha_creacion DESC;`

**Usuarios inactivos por soft delete:** `SELECT id_usuario, nombre_completo, email, estado, deleted_at FROM usuario WHERE estado = 'inactivo' OR deleted_at IS NOT NULL;`

**Contraseñas cifradas:** `SELECT id_usuario, email, password_hash FROM usuario ORDER BY id_usuario;`

**Tokens de recuperación usados:** `SELECT id_reset_token, id_usuario, usado, fecha_creacion, fecha_expiracion FROM password_reset_token ORDER BY id_reset_token DESC;`

**Auditoría reciente:** `SELECT tabla_afectada, operacion, usuario_bd, fecha_evento FROM audit_log ORDER BY fecha_evento DESC LIMIT 20;`

## 11. Procedimiento de control de cambios

Todo cambio debe seguir un proceso controlado para evitar que una corrección pequeña introduzca una falla mayor. El control de cambios también permite explicar técnicamente por qué se tomó una decisión y qué impacto tuvo.

| Etapas         | Descripción                                                 | Salida esperada                                          |
|----------------|-------------------------------------------------------------|----------------------------------------------------------|
| Solicitud      | Se registra cambio, mejora o incidente en el tablero.       | Ticket con descripción, módulo, prioridad y responsable. |
| Análisis       | Se revisa impacto en frontend, backend, BD, seguridad y UX. | Comentario técnico o mini análisis de impacto.           |
| Aprobación     | El líder valida si se trabaja de inmediato o en sprint.     | Cambio aprobado, rechazado o movido al backlog.          |
| Implementación | Se desarrolla en rama separada.                             | Commit descriptivo y código revisable.                   |
| Pruebas        | QA ejecuta casos relacionados y regresión mínima.           | Evidencia de pruebas y resultado.                        |
| Despliegue     | Se publica en ambiente correspondiente.                     | Versión liberada y checklist de despliegue.              |
| Cierre         | Se documenta resultado y lección aprendida.                 | Ticket cerrado con causa y solución.                     |

## 12. Plan de respaldo, restauración y continuidad

La base de datos es el componente más crítico porque contiene usuarios, ligas, vaticinios, resultados, sesiones, auditoría y premios. El mantenimiento debe garantizar que la información pueda recuperarse ante errores humanos, fallas de despliegue o problemas de infraestructura.

| Elemento             | Política definida                                                         |
|----------------------|---------------------------------------------------------------------------|
| Frecuencia de backup | Diaria para producción y semanal para ambiente de pruebas o demostración. |
| Retención            | Conservar al menos los últimos 7 backups diarios y 4 backups semanales.   |
| Formato              | Dump SQL o formato compatible con pg_restore.                             |

| Elemento               | Política definida                                                                                                     |
|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Ubicación              | Repositorio seguro o almacenamiento externo controlado. No subir backups con datos sensibles a repositorios públicos. |
| Prueba de restauración | Mensual o antes de entregas importantes.                                                                              |
| Responsable            | Responsable de base de datos, con apoyo de QA para validar que el sistema funcione después de restaurar.              |

### 12.1 Procedimiento de restauración

24. Notificar al equipo que se iniciará una restauración.
25. Detener temporalmente operaciones que puedan modificar datos.
26. Crear copia de seguridad del estado actual antes de restaurar.
27. Restaurar el backup seleccionado en ambiente de pruebas.
28. Validar tablas críticas: usuario, rol, liga, liga\_miembro, user\_sessions, audit\_log, vaticinio y resultado.
29. Ejecutar pruebas de login, perfil, ligas, vaticinios y cálculos principales.
30. Si la restauración es correcta, aplicar en producción bajo ventana controlada.
31. Registrar fecha, responsable, backup usado y resultado de validación.

### 13. Seguridad, auditoría y trazabilidad

El mantenimiento de seguridad debe ser continuo. En Sistema web de apuestas se deben revisar tanto controles visibles para el usuario como controles internos en base de datos y backend.

| Control                   | Mantenimiento requerido                                                              | Evidencia                                                 |
|---------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Contraseñas               | Verificar que siempre se almacenen como password_hash bcrypt y nunca en texto plano. | Consulta SQL mostrando prefijo \$2b\$ y costo suficiente. |
| JWT y refresh token       | Validar expiración, revocación y limpieza al cerrar sesión.                          | Tabla user_sessions con estado activa/cerrada/expirada.   |
| Roles de aplicación       | Confirmar que administrador y jugador tengan accesos diferenciados.                  | Prueba de /admin/usuarios con rol jugador y admin.        |
| Soft delete               | Comprobar que usuarios, ligas y vaticinios no se eliminen físicamente.               | Registros con deleted_at y estado inactivo.               |
| Audit log                 | Revisar que los triggers registren INSERT, UPDATE y DELETE en tablas obligatorias.   | Consulta a audit_log con valor anterior y nuevo.          |
| Permisos de base de datos | Validar que el usuario de aplicación no tenga permisos DDL.                          | Consulta a pg_roles y documentación en Manual Técnico.    |
| Variables de entorno      | Evitar credenciales reales en Git y mantener .env.example actualizado.               | Revisión de repositorio y archivo .env.example.           |

#### 13.1 Manejo de credenciales

Las credenciales de correo, base de datos y claves de JWT deben gestionarse únicamente mediante variables de entorno. Si una credencial se expone accidentalmente, se debe considerar comprometida y debe rotarse de inmediato.

- No subir archivos .env reales al repositorio.
- Mantener un archivo .env.example sin contraseñas reales.
- Rotar contraseñas de aplicación SMTP si se comparten por error.

- Cambiar SECRET\_KEY si se sospecha exposición.
- Revisar historial de commits antes de publicar el repositorio.

#### 14. Indicadores de seguimiento

Para gestionar el mantenimiento con enfoque de Business Intelligence, se proponen indicadores simples que permiten medir estabilidad, seguridad y capacidad de respuesta.

| Indicador                           | Fórmula / medición                                       | Objetivo                                                |
|-------------------------------------|----------------------------------------------------------|---------------------------------------------------------|
| Tiempo promedio de resolución       | Horas entre reporte y cierre del incidente.              | Reducir el tiempo en incidentes repetitivos.            |
| Porcentaje de incidentes reabiertos | Incidentes reabiertos / incidentes cerrados x 100.       | Mantener por debajo del 10%.                            |
| Sesiones anómalas detectadas        | Cantidad de sesiones concurrentes o vencidas sin cierre. | Identificar patrones sospechosos semanalmente.          |
| Errores de autenticación            | Conteo de 401/403 por semana.                            | Detectar ataques, errores de usuario o fallas de login. |
| Éxito de recuperación de contraseña | Solicitudes completadas / solicitudes generadas.         | Detectar fallos en correo o token.                      |
| Cumplimiento de backup              | Backups ejecutados / backups programados.                | Mantener 100% en producción.                            |
| Cobertura de pruebas de regresión   | Casos ejecutados / casos definidos.                      | Ejecutar 100% antes de release.                         |

#### 15. Matriz de periodicidad

| Actividad                                 | Diario | Semanal | Mensual | Antes de release           | Responsable        |
|-------------------------------------------|--------|---------|---------|----------------------------|--------------------|
| Backup de PostgreSQL                      | Sí     | Sí      | Sí      | Sí                         | BD                 |
| Revisión de logs del backend              |        | Sí      | Sí      | Sí                         | Backend            |
| Revisión de sesiones user_sessions        |        | Sí      | Sí      | Sí                         | Seguridad          |
| Revisión de audit_log                     |        | Sí      | Sí      | Sí                         | BD / Seguridad     |
| Pruebas de regresión                      |        |         |         | Sí                         | QA                 |
| Actualización de dependencias             |        |         | Sí      | Según impacto              | Frontend / Backend |
| Revisión de accesibilidad y responsividad |        |         | Sí      | Sí                         | Frontend / QA      |
| Revisión de credenciales y .env.example   |        |         | Sí      | Sí                         | Seguridad          |
| Prueba de restauración de backup          |        |         | Sí      | Antes de entregas críticas | BD / QA            |

## 16. Registro de actividades de mantenimiento

Cada actividad debe documentarse para conservar trazabilidad y facilitar futuras revisiones. Se recomienda usar el siguiente formato:

| Campo               | Descripción                                            |
|---------------------|--------------------------------------------------------|
| ID de mantenimiento | Código único, por ejemplo MANT-2026-001.               |
| Fecha               | Fecha y hora de inicio y cierre.                       |
| Tipo                | Correctivo, adaptativo, perfectivo o preventivo.       |
| Módulo afectado     | M1 autenticación, M2 ligas, M3 vaticinios, etc.        |
| Descripción         | Qué se realizó y por qué.                              |
| Responsable         | Persona o rol encargado.                               |
| Evidencia           | Capturas, consulta SQL, logs, link de commit o ticket. |
| Resultado           | Exitoso, parcial, fallido o pendiente.                 |
| Observaciones       | Lecciones aprendidas o tareas futuras.                 |

| ID            | Fecha     | Tipo       | Módulo           | Actividad                                                                                                       | Responsable        | Resultado |
|---------------|-----------|------------|------------------|-----------------------------------------------------------------------------------------------------------------|--------------------|-----------|
| MANT-2026-001 | Pendiente | Correctivo | M1 Autenticación | Corrección de cierre de sesión para marcar user_sessions como cerrada.                                          | Backend            | Pendiente |
| MANT-2026-002 | Pendiente | Preventivo | BD / Seguridad   | Validación de password_hash con bcrypt y revisión de soft delete.                                               | Seguridad / BD     | Pendiente |
| MANT-2026-003 | Pendiente | Perfectivo | Frontend         | Mejora visual del perfil y mensajes de éxito.                                                                   | Frontend           | Pendiente |
| MANT-2026-004 | Pendiente | Correctivo | M2 Ligas         | Validación del formulario de creación de ligas y asociación correcta del usuario administrador en liga_miembro. | Backend / BD       | Pendiente |
| MANT-2026-005 | Pendiente | Perfectivo | M2 Ligas         | Mejora del selector de liga en el formulario de invitaciones, mostrando el nombre de la liga en lugar del ID.   | Frontend           | Pendiente |
| MANT-2026-006 | Pendiente | Correctivo | M2 Invitaciones  | Validación del envío de invitaciones por correo y generación correcta del enlace/token de invitación.           | Backend            | Pendiente |
| MANT-2026-007 | Pendiente | Preventivo | M3 Vaticinios    | Revisión de reglas para evitar vaticinios duplicados o registros fuera del tiempo permitido.                    | Backend / BD       | Pendiente |
| MANT-2026-008 | Pendiente | Perfectivo | M3 Vaticinios    | Mejora de mensajes para indicar si un vaticinio fue guardado, actualizado o rechazado.                          | Frontend / Backend | Pendiente |

| ID            | Fecha     | Tipo       | Módulo                          | Actividad                                                                                                        | Responsable            | Resultado |
|---------------|-----------|------------|---------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------|-----------|
| MANT-2026-009 | Pendiente | Adaptativo | M4 Clasificación en tiempo real | Ajuste de cálculo de puntos según reglas del Mundial o cambios definidos por el negocio.                         | Backend                | Pendiente |
| MANT-2026-010 | Pendiente | Preventivo | M4 Clasificación en tiempo real | Validación periódica de resultados, posiciones y puntos generados automáticamente.                               | Backend / QA           | Pendiente |
| MANT-2026-011 | Pendiente | Adaptativo | M5 Administración Mundial       | Carga o actualización de catálogos oficiales del Mundial: países, grupos, sedes, estadios y fases.               | BD / Backend           | Pendiente |
| MANT-2026-012 | Pendiente | Correctivo | M5 Administración Mundial       | Corrección de inconsistencias en catálogos, por ejemplo países sin grupo, estadios sin sede o partidos sin fase. | BD                     | Pendiente |
| MANT-2026-013 | Pendiente | Perfectivo | M6 Premios                      | Mejora del módulo de premios para visualizar ranking, posiciones y premios asignados.                            | Frontend / Backend     | Pendiente |
| MANT-2026-014 | Pendiente | Preventivo | M6 Premios                      | Revisión de reglas de cálculo para evitar asignaciones incorrectas de premios.                                   | Backend / QA           | Pendiente |
| MANT-2026-015 | Pendiente | Correctivo | M7 Panel administrativo         | Corrección de permisos para que solo usuarios autorizados accedan a funciones administrativas.                   | Backend / Seguridad    | Pendiente |
| MANT-2026-016 | Pendiente | Preventivo | M7 Panel administrativo         | Revisión periódica de bitácoras, auditoría y acciones realizadas por administradores.                            | Seguridad / BD         | Pendiente |
| MANT-2026-017 | Pendiente | Preventivo | Base de datos                   | Revisión de llaves foráneas, índices, restricciones y triggers de auditoría.                                     | DBA                    | Pendiente |
| MANT-2026-018 | Pendiente | Correctivo | Base de datos                   | Corrección de errores por columnas faltantes o diferencias entre modelos del backend y tablas reales.            | DBA / Backend          | Pendiente |
| MANT-2026-019 | Pendiente | Preventivo | Despliegue                      | Validación de variables de entorno, conexión a base de datos y configuración CORS.                               | DevOps / Backend       | Pendiente |
| MANT-2026-020 | Pendiente | Perfectivo | Documentación                   | Actualización del manual técnico, manual de usuario y plan de mantenimiento según cambios realizados.            | Documentación / Equipo | Pendiente |

## 17. Riesgos operativos y acciones de mitigación

| Riesgo                                | Impacto                                                  | Mitigación                                                                       |
|---------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------------|
| Credenciales expuestas en repositorio | Acceso no autorizado a BD o correo.                      | Usar .env.example, rotar credenciales y revisar historial Git.                   |
| Sesiones no cerradas correctamente    | Riesgo de accesos persistentes o anomalías.              | Validar logout, expiración y limpieza de user_sessions.                          |
| Falla del servicio SMTP               | No se envían invitaciones ni recuperación de contraseña. | Configurar modo simulado, monitorear errores SMTP y tener proveedor alternativo. |
| Error en cálculo de puntos o premios  | Afecta confianza del sistema y resultados de ligas.      | Pruebas con escenarios conocidos y validación antes del cierre de liga.          |
| Pérdida de datos en PostgreSQL        | Pérdida de usuarios, ligas, vaticinios o resultados.     | Backups automáticos y pruebas de restauración.                                   |
| Cambios sin pruebas                   | Regresiones en funcionalidades ya aprobadas.             | Obligar pruebas de regresión antes de merge y release.                           |
| Dependencias desactualizadas          | Vulnerabilidades o incompatibilidad.                     | Revisión mensual y actualización controlada.                                     |
| Roles mal asignados                   | Usuarios acceden a funciones administrativas.            | Validar permisos desde frontend y backend, y revisar tabla rol.                  |

## 18. Criterios de aceptación posterior al mantenimiento

Ninguna actividad de mantenimiento debe cerrarse sin validar que el sistema funciona correctamente y que no se afectaron flujos críticos.

- El backend levanta sin errores y Swagger muestra los endpoints esperados.
- El frontend compila correctamente y permite navegar por los flujos principales.
- Login, perfil, logout y recuperación de contraseña funcionan si el cambio afecta M1.
- No hay credenciales reales en el repositorio.
- Las consultas de validación en PostgreSQL muestran datos coherentes.
- Las pruebas de regresión relacionadas fueron ejecutadas y documentadas.
- El cambio cuenta con commit descriptivo y evidencia de validación.
- La documentación afectada fue actualizada si el cambio modifica configuración o uso.

## 19. Recomendaciones

El mantenimiento de Sistema web de apuestas debe entenderse como una disciplina continua y no como una actividad aislada al final del proyecto. El sistema maneja usuarios, roles, sesiones, predicciones, premios y datos sensibles, por lo que cada cambio debe revisarse con criterio técnico y de seguridad.

Se recomienda mantener una cultura de evidencia: cada corrección, mejora o revisión preventiva debe dejar rastro en el tablero, en Git y en los documentos de soporte. Esto facilitará la defensa técnica individual, la continuidad del equipo y la confianza en el sistema durante la presentación final.

## 20. Referencias

- Enunciado Proyecto Final Ingeniería de Software. Sistema web de apuestas. Universidad Mariano Gálvez. Documento proporcionado para el proyecto final.
- Documentación oficial de PostgreSQL. Administración, roles, backups, constraints e índices.
- Documentación oficial de FastAPI. Desarrollo de APIs, seguridad y manejo de dependencias.
- Documentación oficial de Angular. Componentes, rutas, formularios y aplicaciones responsivas.
- OWASP Foundation. Recomendaciones generales para seguridad en aplicaciones web y manejo de autenticación.